Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205

Ressurseffektiv programmering

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege feil

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege feil

Ordliste

constructor	konstruktør
dangling pointer	hengande peikar
destructor	destruktør
memory leak	minnelekkasje
ownership	eigarskap
wild pointer	peikar til ingenting

The three most typical memory bugs

1) Access a pointer that was **not initialized**, or that has the value **nullptr**, or that for any other reason points to an **invalid address** in memory.

- **Question:** Why is this dangerous?

2) Memory is **allocated** using new, **but not deallocated again** using delete.

- **Question:** Why is this dangerous?

3) Memory **has been deallocated**: Either it was on the stack in a stack frame that has been removed, or there has been a delete statement. But the address information was stored in **a pointer that still exists**.

- **Question:** Why is this dangerous?

The three most typical memory bugs

1) Access a pointer that was **not initialized**, or that has the value **nullptr**, or that for any other reason points to an **invalid address** in memory. ("**Wild pointer**.")

- **Question:** Why is this dangerous?

2) Memory is **allocated** using new, **but not deallocated again** using delete. This is called a **memory leak**.

- **Question:** Why is this dangerous?

3) Memory **has been deallocated**: Either it was on the stack in a stack frame that has been removed, or there has been a delete statement. But the address information was stored in **a pointer that still exists**: A **dangling pointer**!

- **Question:** Why is this dangerous?

Code that can produce wild pointers

```
#include <iostream>

void crop(int num_strings,  char** strings,  int characters_cropped);

int main(int argc,  char** argv) {
        int jump_to_index = 2;
        crop(argc-1,  argv+1,  jump_to_index);

        for(int i = 1;  i < argc;  i++)
                std::cout  <<  "Argument no. "  <<  i  <<  " was cropped to \""  <<  argv[i]  <<  "\".\n";
}

void crop(int num_strings,  char** strings,  int characters_cropped) {

        for(int i = 0; i < num_strings; i++)
                strings[i] += characters_cropped;
}
```

1. Where is argv in memory? What does it consist of?

Plan: Remove the first 2 characters from each command-line argument.

Implementation: Simply jump ahead by <characters_cropped> char's, that is, here, by 2 characters.

Example file: **wildptr.cpp**

Code that can produce wild pointers

2. What sort of a statement is this? Why do we need it?

```
#include <iostream>

void crop(int num_strings,  char** strings,  int characters_cropped);

int main(int argc,  char** argv)  {
        int jump_to_index = 2;
        crop(argc-1,  argv+1,  jump_to_index);

        for(int i = 1;  i < argc;  i++)
                std::cout  <<  "Argument no. "  <<  i  <<  " was cropped to \""  <<  argv[i]  <<  "\".\n";
}

void crop(int num_strings,  char** strings,  int characters_cropped)  {

        for(int i = 0; i < num_strings; i++)
                strings[i] += characters_cropped;

}
```

3. Why this "-1" and "+1" on argc and argv?

4. How was this supposed to work? When will it inadvertently generate a wild pointer?

Example file: **wildptr.cpp**

used for assert(condition), which checks that condition is true

used for strlen(char str), which returns the length of a C string*

Note that this is <u>one less</u> than the size of str as a char array:
For strlen, the terminal \0 character does not count.

```
#include <cassert>
#include <cstring>
#include <iostream>

void crop(int num_strings,  char** strings,  int characters_cropped);

int main(int argc,  char** argv)  {
        int jump_to_index = 2;
        crop(argc-1,  argv+1,  jump_to_index);

        for(int i = 1;  i < argc;  i++)
                std::cout  <<  "Argument no. "  <<  i  <<  " was cropped to \""  <<  argv[i]  <<  "\".\n";
}

void crop(int num_strings,  char** strings,  int characters_cropped)  {
        assert(characters_cropped >= 0);
        for(int i = 0; i < num_strings; i++)
                if(strlen(strings[i]) >= characters_cropped)
                        strings[i] += characters_cropped;
                else strings[i] += strlen(strings[i]);
}
```

Example file: **wildptr-fixed.cpp**

Hengande peikar: Eksempel frå forrige veke

```
int* global_peikar = nullptr;

void berekning() {
    int d = 5;
    global_peikar = &d;  // no set global_peikar til adressen til d

    std::cout     << "berekning():\t Adressen er " << global_peikar
                  << ", int-verdien er " << *global_peikar << "\n";
}

int main() {
    berekning();  // kalle på funksjon som skal berekne int-verdien

    std::cout     << "main() etterpå:\t Adressen er " << global_peikar
                  << ", int-verdien er " << *global_peikar << "\n";
}
```

Eksempelfil: **levetid.cpp**

How can memory leaks be fixed?

High-level languages operate with *automated garbage collection:* Memory is deallocated when there are no more variables referring to it.

C/C++ memory management on the heap must be done by hand. This causes two possible bugs:

Memory leak: Memory should have been deallocated, but was not.

Dangling pointers: Memory has been deallocated, but it should not.

Discussion:

- What approaches can we try in general, when we have detected a memory leak?
- Brainstorm a list of ideas.
- First look at example code for an extreme case of this.

Minnelekkasje: Eksempel frå forrige veke

```
namespace{
    int* tabell = nullptr;

    void deallokering() {
        assert(tabell != nullptr);

        tabell = nullptr;
    }

    void allokering(std::size_t s) {
        if(tabell != nullptr) deallokering();
            tabell = new int[s];
    }
}
```

Her valde me å tilleggje:
delete[] tabell;

Regelen var: For kvar *new* må vi ha ein tilsvarande *delete*

How can memory leaks be avoided?

Discussion:

- What approaches can we try in general, if we want to decrease the risk of a bug in the form of a memory leak?

There are a few techniques in C++ that help us write safer code with explicit memory management, still done on the heap but less prone to pitfalls.

The key concept for safe manual memory management is **ownership** of a data item, *i.e.*, deciding what entity/part of the code has *responsibility for managing its allocation and deallocation* safely. The entity holding ownership is typically an object – so we will first need to discuss how OOP is done in C++.

Derfor: Tilbake til OOP og klasseimplementering!

The **private and public status of class members** (*i.e.*, properties and methods) is stated in the class definition, where properties and methods are declared:

```
class ExampleClass {
public:
  TypeA getPropertyA() const {return this->propertyA;}
  TypeB* getPropertyB() const {return this->propertyB;}
  void setPropertyA(TypeA a) {this->propertyA = a;}
  void setPropertyA(TypeB* b) {this->propertyB = b;}
  void do_something();

private:
  TypeA propertyA;
  TypeB* propertyB;

  void helper_method();
};
```

Only the public part of the class definition is the interface accessible to code outside the scope of the class.

Typical object-oriented design makes all properties (objects' variables) private. They are read using public "get" methods and modified using public "set" methods.

Methods that are only called by other methods of the same class, but not from outside, are also declared to be private.

Constructors and destructors

Constructor: A method that is called when an object is **allocated**.
Destructor: A method that is (implicitly) called when an object is **deallocated**.

They are not mandatory (as we have seen); use them if you need to specify some functionality for this purpose. Most typically:

- Provide a **constructor** if you want to give the user control over how the private properties of an object are initialized.
- There are also special "copy constructors" and "move constructors". (We will discuss them in detail at a later stage in the course.)
- Provide a **destructor** if your memory management strategy requires it; there might be properties stored as pointers that need to be deleted.

```
class BookIndex {
public:
  BookIndex(int c, int s, int p);
  ~BookIndex();
  …
};
```

```
BookIndex::BookIndex(int c, int s, int p) {
  this->chapter = c; this->section = s; this->page = p;
}
BookIndex::~BookIndex() {
  cout << "Deleting a BookIndex object.\n";
}
```

Constructors and destructors

General rule: For every "new" there must be a matching "delete".

The destructor **T::~T()** is called when an object of type T is deallocated.

This is the case both for objects on the stack and on the heap:

```
void function_name(…)
{
  // constructor is called
  T tobject;
  …
  // destructor is called
  return;
}
```

```
{
  …
  // constructor is called
  T* tpointer = new T;
  …
  // destructor is called
  delete T;
}
```

```
class T
{
public:
  …
  ~T() { delete this->p; }
  …
private:
  S* p …
}
```

If T has ownership over p, this must be done! Without it, there would be a **memory leak**!

There might by other properties that do *not* need to be deallocated manually. (*Why?*)

Constructors and destructors

Core Guidelines:

- **C.20**: "If you can avoid defining default operations, do."
  - "This is known as "the **rule of zero**." Define zero constructors or destructors if it can be done without creating an inconsistent state.
  - A simple and good reason for defining a constructor is to force the user to provide some information that is required.
  - Define a constructor in cases where it does not make sense to initialize the object's properties to some specified default value.

- **C.30**: "Define a destructor if a class needs an explicit action at object destruction." And related, **C.31**: "All resources acquired by a class must be released by the class's destructor."
  - If a data structure needs to be built up (memory allocated, *etc.*), this normally requires both a constructor and a destructor.
  - For such cases, we will learn the *rule of three* and the *rule of five*.

Glossar

Konstruktør

Definisjon: metode som blir kalla på idet eit objekt blir allokert

Destruktør

Definisjon: metode som blir kalla på idet eit objekt blir deallokert

Glossar

Peikar til ingenting

Definisjon: peikar med ein verdi som ikkje er ei gyldig minneadresse

- Den ugyldige verdien er oftast **nullptr**, dvs. minneadressa 0x0000000000000000. Denne verdien blir ofte tildelt med hensikt, for å markere at peikaren ikkje peikar på noko dataelement.
- Peikaren skal ikkje derefererast så lenge verdien er nullptr; bruk ein if-klausul, **if(p != nullptr) { … }**.
- Feil peikararitmetikk kan vera ei anna årsak til at det oppstår ein peikar til ingenting.

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

Problemliste B

INF205

19. februar 2026

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205

Ressurseffektiv programmering

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege feil